The basics of Kotlin Multiplatform project structure

Edit page 01 April 2026

With Kotlin Multiplatform, you can share code among different platforms. This article explains the constraints of the shared code, how to distinguish between shared and platform-specific parts of your code, and how to specify the platforms on which this shared code works.

You'll also learn the core concepts of Kotlin Multiplatform project setup, such as common code, targets, platform-specific and intermediate source sets, and test integration. That will help you set up your multiplatform projects in the future.

The model presented here is simplified compared to the one used by Kotlin. However, this basic model should be adequate for the majority of cases.

Common code

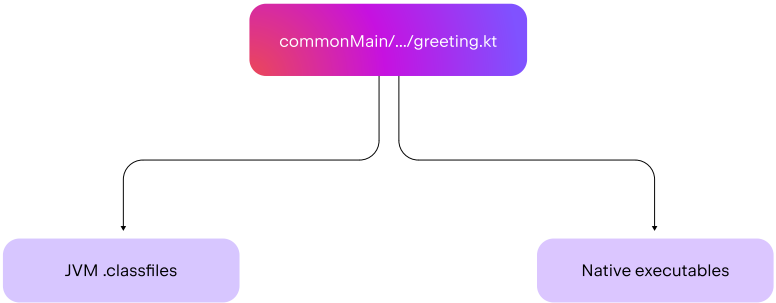
Common code is the Kotlin code shared among different platforms.

Consider the simple "Hello, World" example:

```
fun greeting() {
    println("Hello, Kotlin Multiplatform!")
}
```

Kotlin code shared among platforms is typically located in the `commonMain` directory. The location of code files is important, as it affects the list of platforms to which this code is compiled.

The Kotlin compiler gets the source code as input and produces a set of platform-specific binaries as a result. When compiling multiplatform projects, it can produce multiple binaries from the same code. For example, the compiler can produce JVM `.class` files and native executable files from the same Kotlin file:



Not every piece of Kotlin code can be compiled to all platforms. The Kotlin compiler prevents you from using platform-specific functions or classes in your common code if this code can't be compiled to a different platform.

For instance, you can't use the `java.io.File` dependency from the common code. It's a part of the JDK, while common code is also compiled to native code, where the JDK classes are not available:

```
6
7   fun greeting() {
8       java.io.File("greeting.kt").writeText("Hello, Multiplatform!")
9   }
10
```

Unresolved reference: java

Create local variable 'java' ⌥⇧⏎ More actions... ⌥⏎

In common code, you can use Kotlin Multiplatform libraries. These libraries provide a common API that can be implemented differently on different platforms. In this case, platform-specific APIs serve as extra parts, and trying to use such an API in common code results in an error.

For example, `kotlinx.coroutines` is a Kotlin Multiplatform library that supports all targets, but it also has a platform-specific part that converts `kotlinx.coroutines` concurrent primitives to JDK concurrent primitives, like `fun CoroutinesDispatcher.asExecutor(): Executor`. This additional part of the API isn't available in `commonMain`.

Targets

Targets define the platforms to which Kotlin compiles the common code. These could be, for example, the JVM, JS, Android, iOS, or Linux. The previous example compiled the common code to the JVM and native targets.

A **Kotlin target** is an identifier that describes a compilation target. It defines the format of the produced binaries, available language constructions, and allowed dependencies.

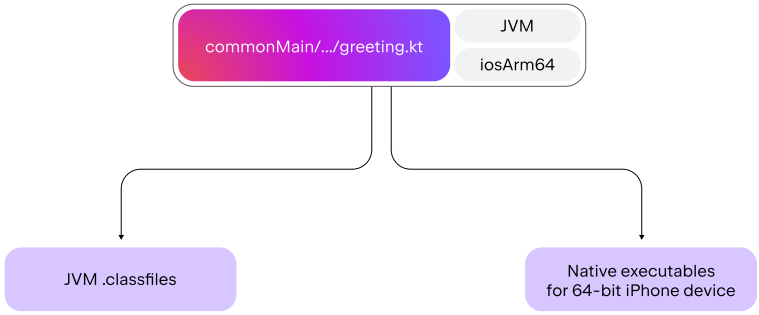
> Targets can also be referred to as platforms. See the full list of supported targets.

You should first **declare** a target to instruct Kotlin to compile code for that specific target. In Gradle, you declare targets using predefined DSL calls inside the `kotlin {}` block:

```
kotlin {
    jvm() // Declares a JVM target
    iosArm64() // Declares a target that corresponds to 64-bit iPhones
}
```

This way, each multiplatform project defines a set of supported targets. See the Hierarchical project structure section to learn more about declaring targets in your build scripts.

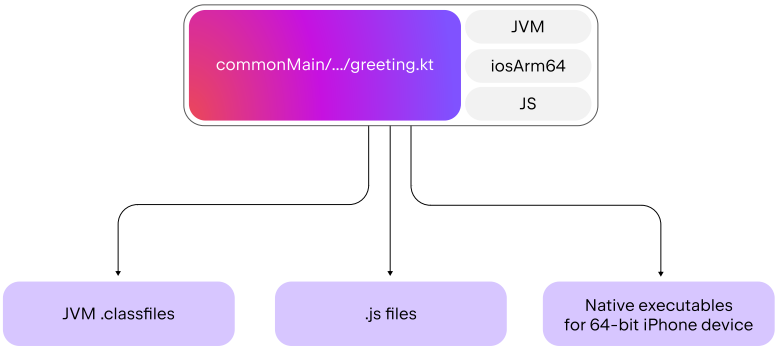
With the `jvm` and `iosArm64` targets declared, the common code in `commonMain` will be compiled to these targets:



To understand which code is going to be compiled to a specific target, you can think of a target as a label attached to Kotlin source files. Kotlin uses these labels to determine how to compile your code, which binaries to produce, and which language constructions and dependencies are allowed in that code.

> If your project only has a single target (for example, JVM), you can access target-specific symbols with appropriate visibility from common code. However, as soon as you add a second target, target-specific symbols become unavailable in common code. Keep this limitation in mind during migrations and other intermediary project states.

If you want to compile the `greeting.kt` file to `.js` as well, you only need to declare the JS target. The code in `commonMain` then receives an additional `js` label, corresponding to the JS target, which instructs Kotlin to produce `.js` files:



That's how the Kotlin compiler works with the common code compiled to all the declared targets. See Source sets to learn how to write platform-specific code.

Source sets

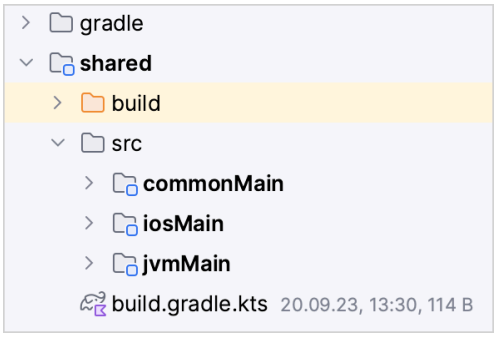
A **Kotlin source set** is a set of source files with its own targets, dependencies, and compiler options. It's the main way to share code in multiplatform projects.

Each source set in a multiplatform project:

- Has a name that is unique for a given project.
- Contains a set of source files and resources, usually stored in the directory with the name of the source set.
- Specifies a set of targets to which the code in this source set compiles. These targets impact which language constructions and dependencies are available in this source set.
- Defines its own dependencies and the compiler options.

Kotlin provides a bunch of predefined source sets. One of them is `commonMain`, which is present in all multiplatform projects and compiles to all declared targets.

You interact with source sets as directories inside `src` in Kotlin Multiplatform projects. For example, a project with the `commonMain`, `iosMain`, and `jvmMain` source sets has the following structure:



In Gradle scripts, you access source sets by name inside the `kotlin.sourceSets {}` block:

```
kotlin {
    // Targets declaration:
    // …

    // Source set declaration:
    sourceSets {
        commonMain {
            // Configure the commonMain source set
        }
    }
}
```

Aside from `commonMain`, other source sets can be either platform-specific or intermediate.

Platform-specific source sets

While having only common code is convenient, it's not always possible. Code in `commonMain` compiles to all declared targets, and Kotlin doesn't allow you to use any platform-specific APIs there.

In a multiplatform project with native and JS targets, the following code in `commonMain` doesn't compile:

```
// commonMain/kotlin/common.kt
// Doesn't compile in common code
fun greeting() {
    java.io.File("greeting.txt").writeText("Hello, Multiplatform!")
}
```

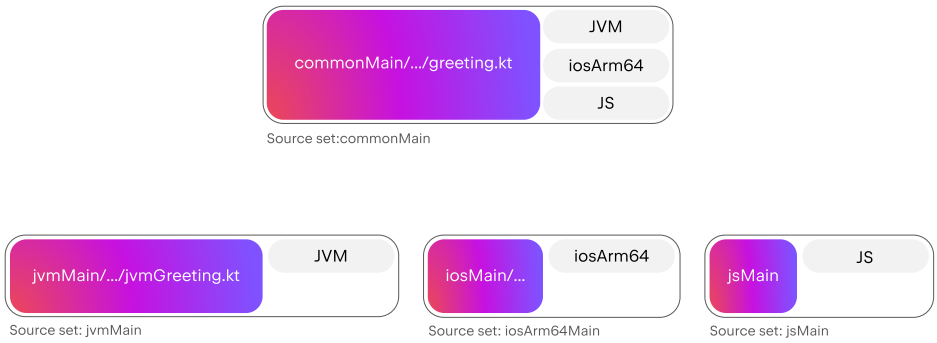
As a solution, Kotlin creates platform-specific source sets, also referred to as platform source sets. Each target has a corresponding platform source set that compiles for only that target. For example, a `jvm` target has the corresponding `jvmMain` source set that compiles to only the JVM. Kotlin allows using platform-specific dependencies in these source sets, for instance, JDK in `jvmMain`:

```kotlin
// jvmMain/kotlin/jvm.kt
// You can use Java dependencies in the `jvmMain` source set
fun jvmGreeting() {
    java.io.File("greeting.txt").writeText("Hello, Multiplatform!")
}
```

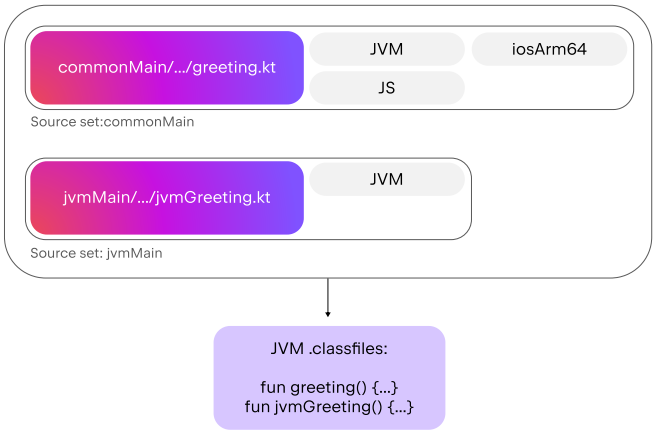
Compilation to a specific target

Compilation to a specific target works with multiple source sets. When Kotlin compiles a multiplatform project to a specific target, it collects all source sets labeled with this target and produces binaries from them.

Consider an example with `jvm`, `iosArm64`, and `js` targets. Kotlin creates the `commonMain` source set for common code and the corresponding `jvmMain`, `iosArm64Main`, and `jsMain` source sets for specific targets:



During compilation to the JVM, Kotlin selects all source sets labeled with "JVM", namely, `jvmMain` and `commonMain`. It then compiles them together to the JVM class files:



Because Kotlin compiles `commonMain` and `jvmMain` together, the resulting binaries contain declarations from both `commonMain` and `jvmMain`.

When working with multiplatform projects, remember:

- If you want Kotlin to compile your code to a specific platform, declare a corresponding target.
- To choose a directory or source file to store the code, first decide among which targets you want to share your code:
  - If the code is shared among all targets, it should be declared in `commonMain`.
  - If the code is used for only one target, it should be defined in a platform-specific source set for that target (for example, `jvmMain` for the JVM).
- Code written in platform-specific source sets can access declarations from the common source set. For example, the code in `jvmMain` can use code from `commonMain`. However, the opposite isn't true: `commonMain` can't use code from `jvmMain`.
- Code written in platform-specific source sets can use the corresponding platform dependencies. For example, the code in `jvmMain` can use Java-only libraries, like Guava ↗ or Spring ↗.

Intermediate source sets

Simple multiplatform projects usually have only common and platform-specific code. The `commonMain` source set represents the common code shared among all declared targets. Platform-specific source sets, like `jvmMain`, represent platform-specific code compiled only to the respective target.

In practice, you often need more granular code sharing.

Consider an example where you need to target all modern Apple devices and Android devices:

```
kotlin {
    android()
    iosArm64()   // 64-bit iPhone devices
    macosArm64() // Modern Apple Silicon-based Macs
    watchosArm64() // Modern 64-bit Apple Watch devices
    tvosArm64()  // Modern Apple TV devices
}
```

And you need a source set to add a function that generates a UUID for all Apple devices:

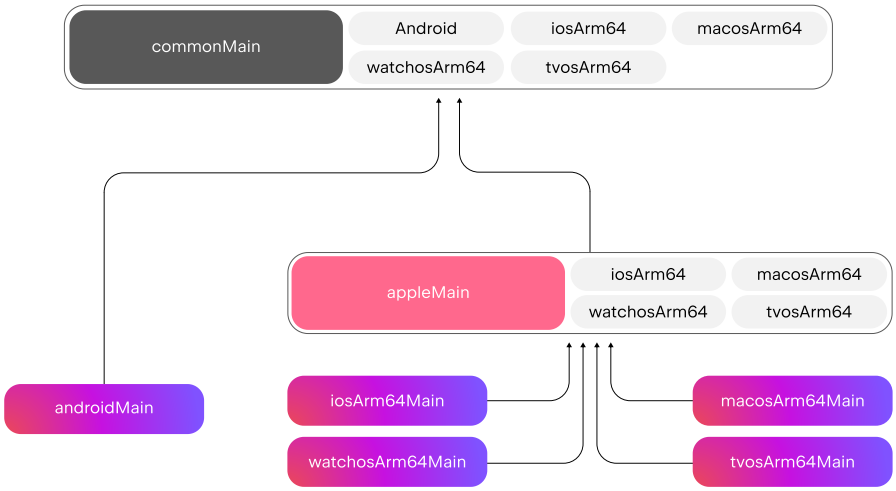
```
import platform.Foundation.NSUUID

fun randomUuidString(): String {
    // You want to access Apple-specific APIs
    return NSUUID().UUIDString()
}
```

You can't add this function to `commonMain`. `commonMain` is compiled to all declared targets, including Android, but `platform.Foundation.NSUUID` is an Apple-specific API that's not available on Android. Kotlin shows an error if you try to reference `NSUUID` in `commonMain`.

You could copy and paste this code to each Apple-specific source set: `iosArm64Main`, `macosArm64Main`, `watchosArm64Main`, and `tvosArm64Main`. But this approach is not recommended because duplicating code like this is prone to errors.

To solve this issue, you can use **intermediate source sets**. An intermediate source set is a Kotlin source set that compiles to some, but not all of the targets in the project. You can also see intermediate source sets referred to as hierarchical source sets or simply hierarchies.

Kotlin creates some intermediate source sets by default. In this specific case, the resulting project structure will look like this:

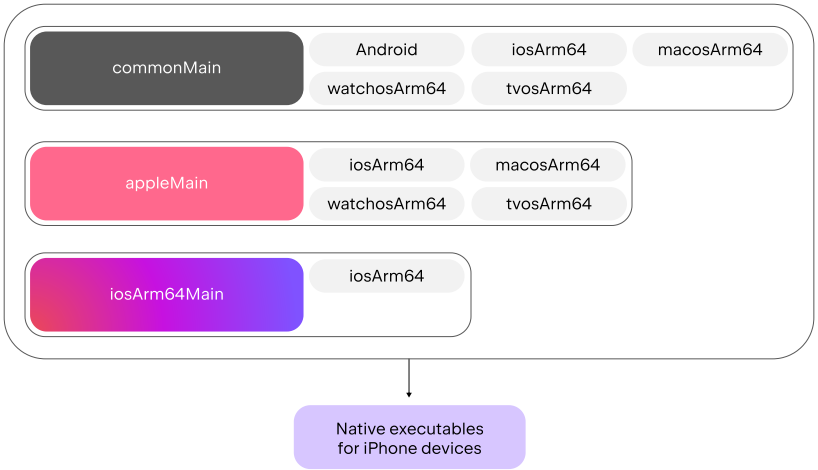


Here, the multicolored blocks at the bottom are platform-specific source sets. Target labels are omitted for clarity.

The `appleMain` block is an intermediate source set created by Kotlin for sharing code compiled to Apple-specific targets. The `appleMain` source set compiles to only Apple targets. Therefore, Kotlin allows using Apple-specific APIs in `appleMain`, and you can add the `randomUUID()` function here.

> See Hierarchical project structure to find all intermediate source sets that Kotlin creates and sets up by default and learn what you should do if Kotlin doesn't provide the intermediate source set you need by default.

During compilation to a specific target, Kotlin gets all of the source sets, including intermediate source sets, labeled with this target. Therefore, all the code written in the `commonMain`, `appleMain`, and `iosArm64Main` source sets is combined during compilation to the `iosArm64` platform target:



It's okay if some source sets don't have sources. For example, in iOS development, there's usually no need to provide code that is specific for iOS devices but not for iOS simulators. `iosArm64Main` is therefore rarely used.

Apple device and simulator targets

Integration with tests

Real-life projects also require tests alongside the main production code. This is why all source sets created by default have the `Main` and `Test` suffixes. `Main` contains production code, while `Test` contains tests for this code. The connection between them is established automatically, and tests can use the API provided by the `Main` code without additional configuration.

The `Test` counterparts are also source sets similar to `Main`. For example, `commonTest` is a counterpart for `commonMain` and compiles to all of the declared targets, allowing you to write common tests. Platform-specific test source sets, such as `jvmTest`, are used to write platform-specific tests, for example, JVM-specific tests or tests that need JVM APIs.

Besides having a source set to write your common test, you also need a multiplatform testing framework. Kotlin provides a default `kotlin.test` ↗ library that comes with the `@kotlin.Test` annotation and various assertion methods like `assertEquals` and `assertTrue`.

You can write platform-specific tests like regular tests for each platform in their respective source sets. Like with the main code, you can have platform-specific dependencies for each source set, such as `JUnit` for JVM and `XCTest` for iOS. To run tests for a particular target, use the `<targetName>Test` task.

Learn how to create and run multiplatform tests in the Test your multiplatform app tutorial.

What's next?

- Learn more about declaring and using predefined source sets in Gradle scripts
- Explore advanced concepts of the multiplatform project structure
- Learn more about target compilation and creating custom compilations